



# UANL

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN



## FCFM

### **Análisis Numérico**

**M.A. Maria Del Carmen Martínez Cejudo**

### **Proyecto 2 Bisección**

**Sebastián Calderón Carrillo 2087472**

**Edson Armando Amaya Macias 2151976**

**Juan José Trujillo Garza 1972407**

**Grupo 031**

## Introducción

El método de bisección, también conocido como método de búsqueda binaria o de Bolzano, es un algoritmo numérico para encontrar raíces de funciones continuas. Se basa en el teorema del valor intermedio: si una función continua cambia de signo en los extremos de un intervalo  $[a, b]$ , entonces existe al menos una raíz en dicho intervalo. El método divide repetidamente el intervalo a la mitad y selecciona el subintervalo donde ocurre el cambio de signo, garantizando convergencia hacia una raíz con una precisión controlada por una tolerancia dada.

A diferencia de otros métodos como Newton-Raphson, la bisección no requiere derivadas y siempre converge para funciones continuas con cambio de signo, aunque su convergencia es más lenta (lineal). Es un método robusto y fácil de implementar, ideal para obtener una primera aproximación de la raíz.

Para la implementación en Python se dividió el proyecto en módulos con responsabilidades separadas: ***modules/biseccion.py*** concentra la lógica matemática (evaluación de polinomios, algoritmo de bisección y generación del historial de iteraciones), ***modules/utils.py*** se encarga de la validación y lectura de datos por consola, y ***modules/menu.py*** implementa el menú interactivo que conecta ambos módulos.

Como decisión de diseño clave, se utiliza una lista de coeficientes ordenados de mayor a menor grado para representar el polinomio, y se evalúa mediante la función *evaluar\_polimonia*, que aplica la suma de términos  $c * x^{\text{grado}}$ . El núcleo del programa es la función *biseccion*, que recibe los coeficientes, los límites del intervalo, la tolerancia y el número máximo de iteraciones, y devuelve la raíz aproximada, el número de iteraciones realizadas y una lista con el historial completo de cada iteración. Antes de aplicar el método, se verifica que exista un cambio de signo en el intervalo; si no lo hay, se lanza una excepción con un mensaje claro. Toda la entrada del usuario se valida con manejo de excepciones en *utils.py*, pidiendo nuevamente el dato en caso de error.

## Codificación

En esta sección se presentan y explican las funciones implementadas en el código, destacando cómo cada una traduce las propiedades matemáticas del método de bisección y cómo se integran para formar el flujo completo del programa.

### *evaluar\_polimONIO(coeficientes, x)*

```
99  def evaluar_polimONIO(coeficientes: List[float], x: float) -> float:
100      """Evalúa un polinomio en un punto x dado sus coeficientes.
101      Los coeficientes se ordenan de mayor a menor grado.
102      Ejemplo: [1, -3, 2] representa  $x^2 - 3x + 2$ 
103
104      Args:
105          coeficientes: Coeficientes del polinomio.
106          x: Valor de x a evaluar en el polinomio.
107
108      Returns:
109          float: Valor de la función evaluada en x.
110      """
111      resultado = 0.0
112      n = len(coeficientes)
113      for i, c in enumerate(coeficientes):
114          grado = n - 1 - i
115          resultado += c * (x ** grado)
116      return resultado
```

Explicación:

Esta función evalúa un polinomio en un punto  $x$  dado. Recibe una lista de coeficientes ordenados de mayor a menor grado y un valor  $x$ . Recorre los coeficientes y, para cada uno, calcula  $c * (x ** grado)$ , donde el grado se obtiene como  $n - 1 - i$  (siendo  $n$  la cantidad de coeficientes). Suma todos los términos y devuelve el resultado. Es la base para calcular  $f(a)$ ,  $f(b)$  y  $f(c)$  en cada iteración del método.

biseccion(coeficientes, lim\_izq, lim\_der, tolerancia=1e-6, max\_iter=100)

```
def biseccion(coeficientes: List[float], lim_izq: float, lim_der: float,
              tolerancia: float = 1e-6, max_iter: int = 100) -> Tuple[float, int, List[Tuple]]:
    """Implementa el método de bisección para encontrar una raíz.

    Args:
        coeficientes: Función continua
        lim_izq: Extremo izquierdo del intervalo
        lim_der: Extremo derecho del intervalo
        tolerancia: Tolerancia (criterio de parada)
        max_iter: Número máximo de iteraciones

    Returns:
        Tuple[float, int, List[Tuple]]: Tupla con (raíz, iteraciones, historial)

    Raises:
        ValueError: Si  $f(a) * f(b) \geq 0$  (no hay cambio de signo)
    """

    f_lim_izq = evaluar_polimonia(coeficientes, lim_izq)
    f_lim_der = evaluar_polimonia(coeficientes, lim_der)

    # Check if one of the fa and fb is actually a solution

    if f_lim_izq * f_lim_der >= 0:
        raise ValueError("No hay cambio de signo en el intervalo [a, b]. No se puede aplicar bisección.")

    historial = []

    for i in range(0, max_iter):
        punto_medio = (lim_izq + lim_der) / 2
        f_punto_medio = evaluar_polimonia(coeficientes, punto_medio)

        historial.append((i+1, lim_izq, lim_der, punto_medio, f_lim_izq, f_lim_der, f_punto_medio))

        if abs(f_punto_medio) <= tolerancia or (lim_der - lim_izq) / 2 <= tolerancia:
            return punto_medio, i, historial

        if f_lim_izq * f_punto_medio < 0:
            lim_der = punto_medio
            f_lim_der = f_punto_medio
        else:
            lim_izq = punto_medio
            f_lim_izq = f_punto_medio

    return punto_medio, max_iter, historial
```

Explicación:

Esta es la función principal del método. Primero evalúa el polinomio en los extremos `lim_izq` y `lim_der`. Si el producto  $f(lim\_izq) * f(lim\_der) \geq 0$ , lanza una excepción `ValueError` indicando que no hay cambio de signo y que no se puede aplicar bisección. Si hay cambio de signo, inicializa una lista `historial` y comienza un ciclo de hasta `max_iter` iteraciones. En cada iteración:

1. *Calcula el punto medio  $punto\_medio = (lim\_izq + lim\_der) / 2$ .*
2. *Evalúa  $f(punto\_medio)$ .*
3. *Agrega una tupla al historial con el número de iteración, los límites actuales, el punto medio, y los valores de  $f$  en los extremos y en el punto medio.*
4. *Verifica el criterio de parada: si  $abs(f(punto\_medio)) \leq tolerancia$  o si la mitad del ancho del intervalo  $(lim\_der - lim\_izq) / 2 \leq tolerancia$ , devuelve el punto medio, el número de iteraciones realizadas (índice  $i$ ) y el historial.*
5. *Si no se cumple el criterio, decide si la raíz está en  $[lim\_izq, punto\_medio]$  o en  $[punto\_medio, lim\_der]$  según el signo de  $f(lim\_izq) * f(punto\_medio)$ , y actualiza los límites y los valores de  $f$  correspondientes.*

Si se alcanza el número máximo de iteraciones sin cumplir la tolerancia, devuelve el último punto medio calculado,  $max\_iter$  y el historial completo. Esta función garantiza que el intervalo siempre contenga una raíz y que el error se reduzca a la mitad en cada paso.

## ejecutar\_biseccion()

```
def ejecutar_biseccion():
    """Ejecuta el método de bisección con entrada del usuario."""
    print("\n--- Método de Bisección ---\n")

    # Leer coeficientes del polinomio
    print("Ingresa los coeficientes del polinomio (de mayor a menor grado)")
    print("Ejemplo: para  $x^2 - 3x + 2$ , ingresa: 1 -3 2")
    entrada = input("Coeficientes: ").strip().split()

    try:
        coeficientes = [float(c) for c in entrada]
    except ValueError:
        print("Error: Los coeficientes deben ser números.")
        return

    if len(coeficientes) == 0:
        print("Error: Debes ingresar al menos un coeficiente.")
        return

    # Leer intervalo [a, b]
    lim_izq = leer_real("Extremo izquierdo (a): ")
    lim_der = leer_real("Extremo derecho (b): ")

    if lim_izq >= lim_der:
        print("Error: 'a' debe ser menor que 'b'.")
        return

    # Leer tolerancia
    tolerancia = leer_real("Tolerancia (ej. 0.000001): ")

    try:
        raiz, iteraciones, historial = biseccion(coeficientes, lim_izq, lim_der, tolerancia)

        # Mostrar resultados
        print("\n=== Resultados ===")
        print(f"Raíz encontrada: {raiz}")
        print(f"Iteraciones: {iteraciones}")

        # Mostrar tabla de iteraciones
        print("\n--- Tabla de iteraciones ---")
        print(f"{'Iter':>4} {'a':>12} {'b':>12} {'c':>12} {'f(c)':>12}")
        print("-" * 56)
        for i, a_val, b_val, c, fa, fb, fc in historial:
            print(f"{i:>4} {a_val:>12.6f} {b_val:>12.6f} {c:>12.6f} {fc:>12.6f}")
    except ValueError as error:
        print(f"Error: {error}")
```

Explicación:

Esta función maneja la interacción con el usuario para el método de bisección. Muestra un mensaje explicativo, solicita los coeficientes del polinomio (separados por espacios, de mayor a menor grado), los límites izquierdo y derecho del intervalo, y la tolerancia. Valida que los coeficientes sean números reales y que al menos haya uno. También verifica que  $lim_{izq} < lim_{der}$ . Luego llama a la función `biseccion` y, si no hay errores, muestra la raíz encontrada, el número de iteraciones y una tabla formateada con el historial de iteraciones (columnas: lter, a, b, c, f(c)). Si ocurre un `ValueError` (por ejemplo, por falta de cambio de signo), lo captura y muestra el mensaje de error. Esta función conecta la entrada del usuario con la lógica matemática y presenta los resultados de forma clara.

## utils.py (funciones de utilidad)

Explicación:

Este módulo agrupa funciones auxiliares para la entrada/salida por consola:

- `leer_opcion(prompt, lower, upper)`: lee un entero dentro de un rango cerrado, repitiendo si hay error.

```
def leer_opcion(prompt:str, lower:int, upper:int) -> int:
    """Lee un entero dentro del rango cerrado [lower, upper]."""
    while True:
        entrada = input(prompt).strip()
        try:
            valor = int(entrada)
            if lower <= valor <= upper:
                return valor
            print(f"Opción fuera de rango (se esperaba entre {lower} y {upper}).")
        except ValueError:
            print(f'"{entrada}" no es un número entero válido.')
```

- `leer_entero(prompt)`: lee un entero cualquiera.

```
def leer_entero(prompt: str) -> int:
    """Lee un número entero del usuario."""
    while True:
        entrada = input(prompt).strip()
        try:
            return int(entrada)
        except ValueError:
            print(f'"{entrada}" no es un número entero válido.')
```

- `leer_real(prompt)`: lee un número real (float), repitiendo si hay error.

```
def leer_real(prompt: str) -> float:
    """Lee un número real del usuario."""
    while True:
        entrada = input(prompt).strip()
        try:
            return float(entrada)
        except ValueError:
            print(f'"{entrada}" no es un número real válido.')
```

- *borrar\_consola(): limpia la pantalla según el sistema operativo (Windows: cls, otros: clear), pero solo si la entrada es una terminal interactiva (sys.stdin.isatty()).*

```
def borrar_consola():
    """Detecta el sistema operativo y borra la consola."""
    if not sys.stdin.isatty():
        return
    os.system("cls" if os.name == "nt" else "clear")
```

- *pausar(): simula el comando pause de Windows o espera un Enter en otros sistemas, también solo en terminal interactiva.*

```
def pausar():
    """Simula el comportamiento de pause en cmd."""
    if not sys.stdin.isatty():
        return
    if os.name == "nt":
        os.system("pause")
    else:
        input("Presiona Enter para continuar...")
```

Estas funciones garantizan que el programa no falle por entradas inválidas y que la experiencia de usuario sea fluida.

## Integración de los módulos

El flujo completo del programa es el siguiente:

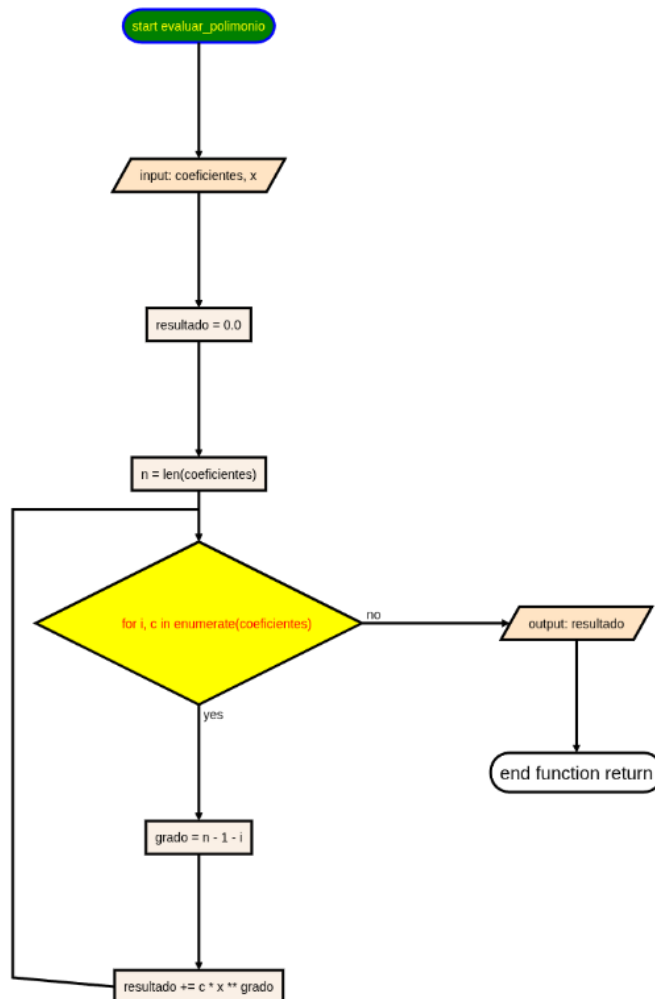
1. main.py (o el script de ejecución) llama a run() del módulo menu.
2. run() muestra el menú y, si se elige la opción 1, llama a ejecutar\_biseccion() del módulo biseccion.
3. ejecutar\_biseccion() solicita los datos al usuario usando leer\_real de utils, valida los coeficientes y el intervalo, y llama a biseccion().

4. `biseccion()` utiliza `evaluar_polimonio()` para calcular los valores de  $f$  y ejecuta el algoritmo, devolviendo la raíz, el número de iteraciones y el historial.
5. `ejecutar_biseccion()` muestra los resultados y la tabla de iteraciones.
6. `run()` pausa la pantalla y vuelve al menú hasta que el usuario elija salir.

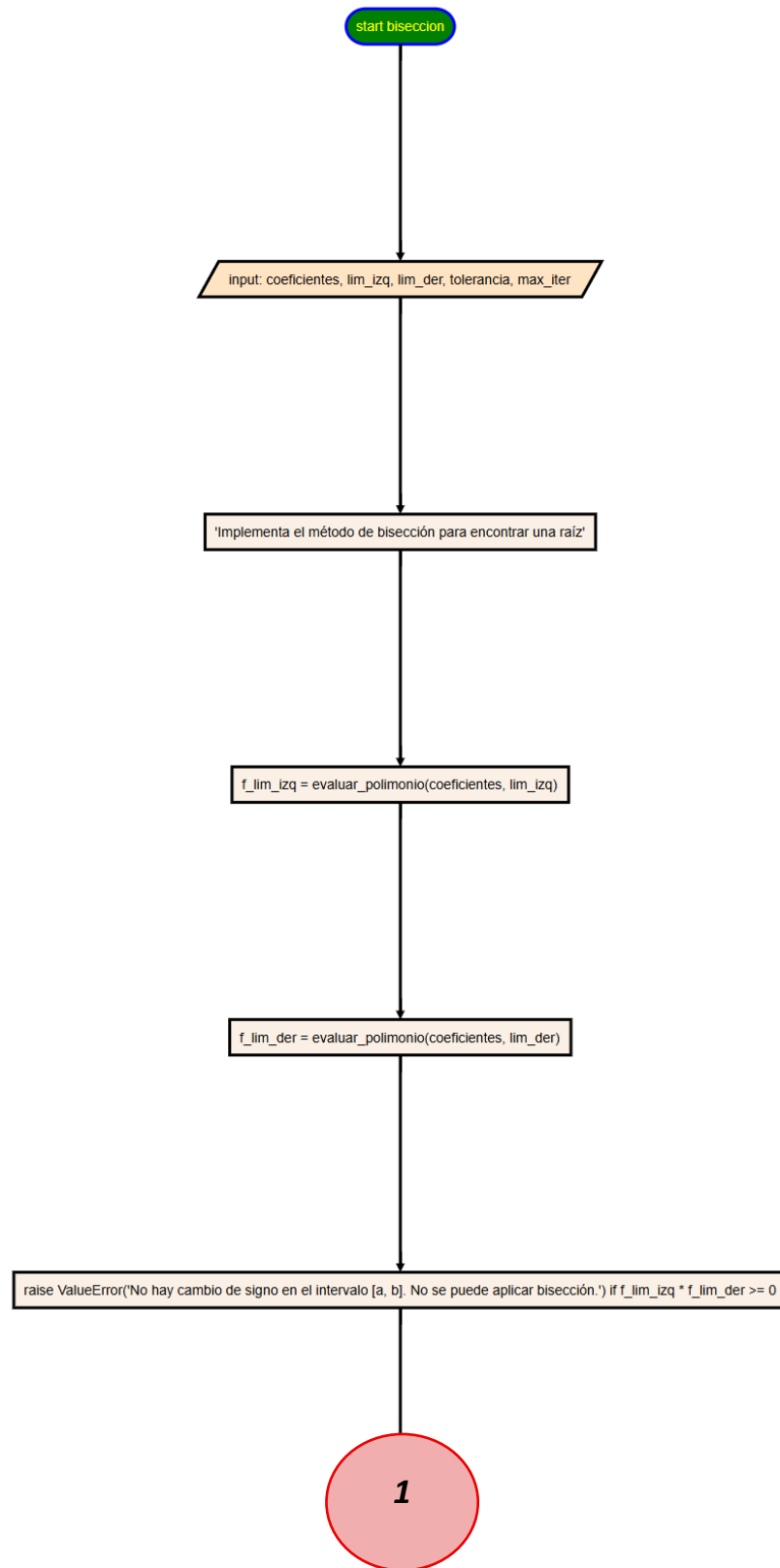
Esta separación en módulos facilita el mantenimiento, las pruebas unitarias y la reutilización del código en otros proyectos.

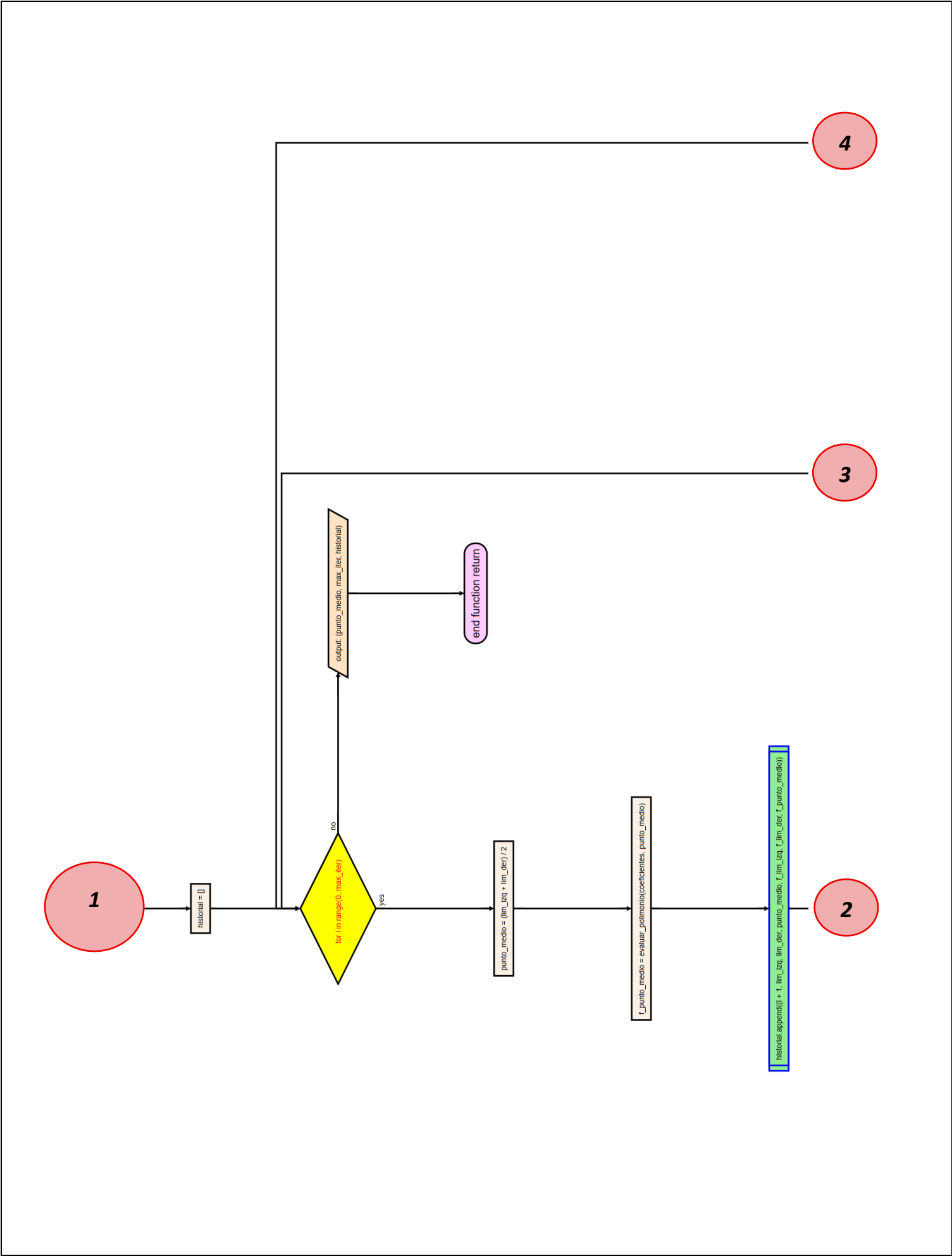
## Diagramas de flujo

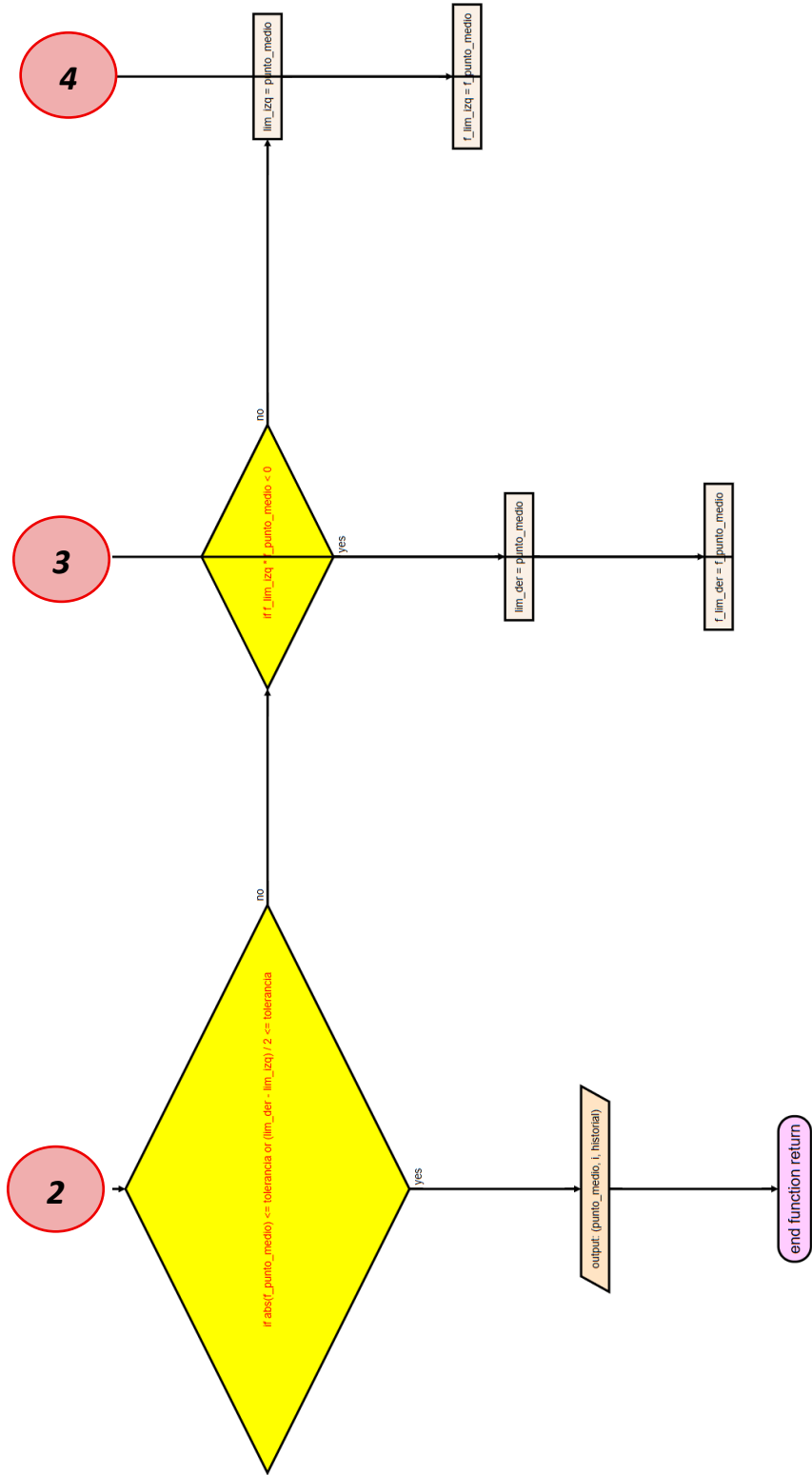
### Evaluar Polinomio



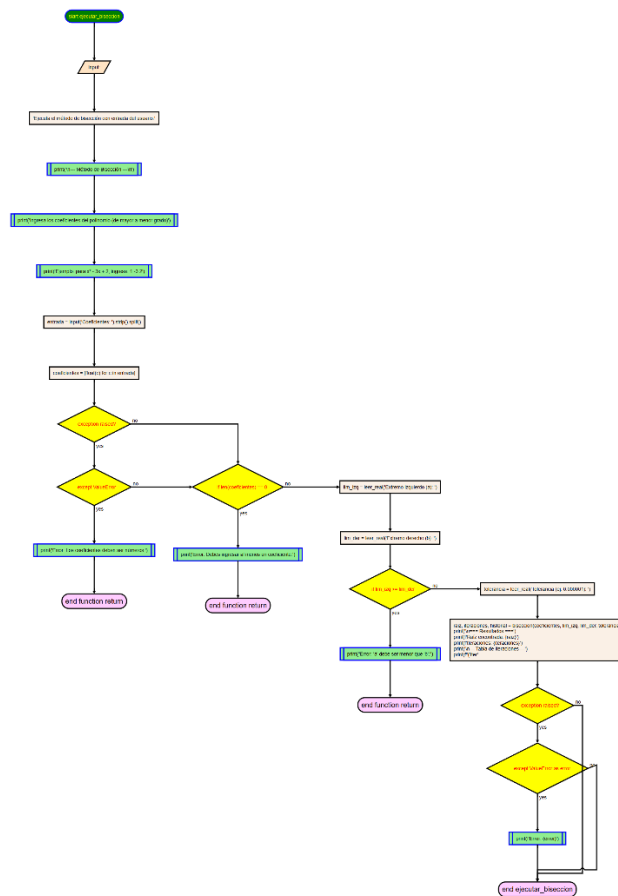
## Bisección







## Ejecutar bisección

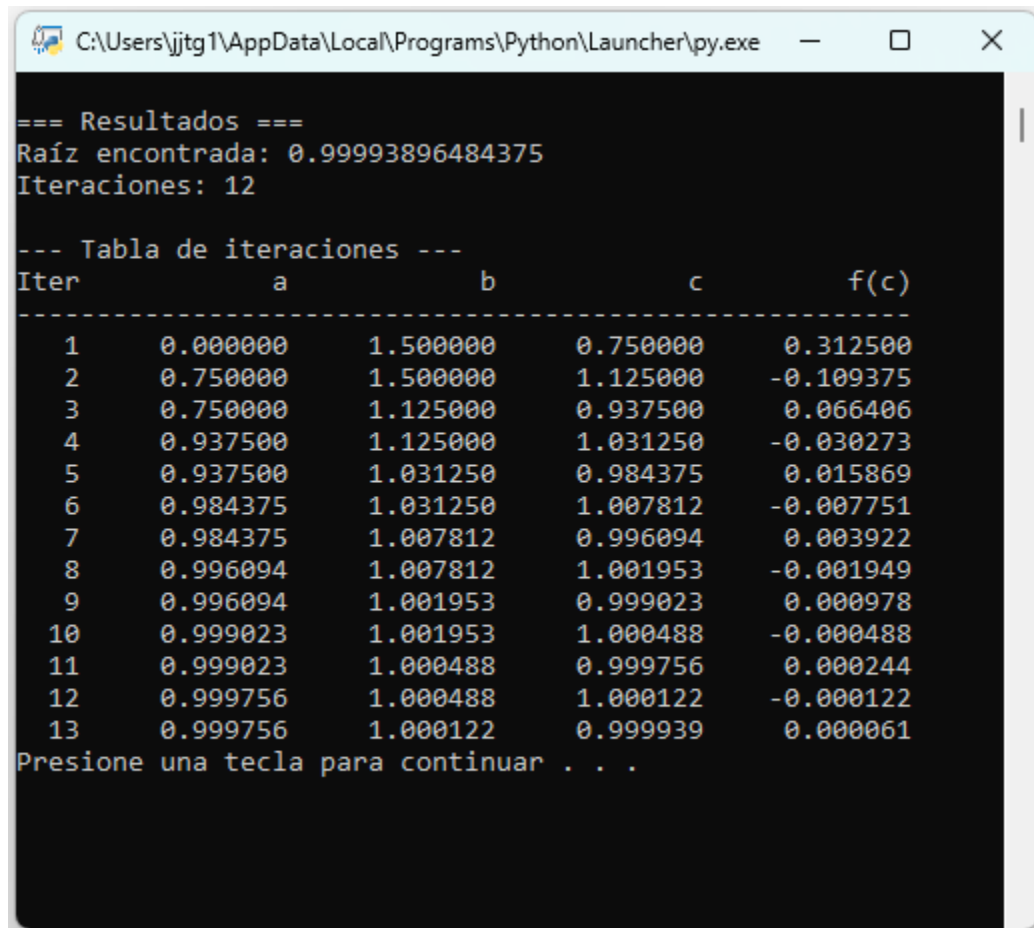


## Conclusión general

El desarrollo de este proyecto permitió consolidar tanto la comprensión matemática como la aplicación práctica del Método de Bisección en la búsqueda de raíces de ecuaciones polinomiales. A través de la implementación algorítmica basada en el Teorema del Valor Intermedio (o Teorema de Bolzano), se comprobó la alta fiabilidad y la convergencia garantizada del método cuando se asegura un cambio de signo en los extremos del intervalo inicial. Si bien su convergencia es de carácter lineal y requiere un número mayor de iteraciones en comparación con métodos de mayor orden, su estabilidad lo convierte en una técnica fundamental e indispensable dentro del análisis numérico.

La automatización del algoritmo no solo elimina el margen de error humano y optimiza el cálculo iterativo de polinomios, sino que también reafirma el valor de integrar conceptos matemáticos rigurosos con buenas prácticas de programación para la resolución eficiente de problemas científicos e ingenieriles

## Ejemplo de ejecución



```
=== Resultados ===  
Raíz encontrada: 0.99993896484375  
Iteraciones: 12  
  
--- Tabla de iteraciones ---  
Iter      a          b          c          f(c)  
-----  
1      0.000000      1.500000      0.750000      0.312500  
2      0.750000      1.500000      1.125000     -0.109375  
3      0.750000      1.125000      0.937500      0.066406  
4      0.937500      1.125000      1.031250     -0.030273  
5      0.937500      1.031250      0.984375      0.015869  
6      0.984375      1.031250      1.007812     -0.007751  
7      0.984375      1.007812      0.996094      0.003922  
8      0.996094      1.007812      1.001953     -0.001949  
9      0.996094      1.001953      0.999023      0.000978  
10     0.999023      1.001953      1.000488     -0.000488  
11     0.999023      1.000488      0.999756      0.000244  
12     0.999756      1.000488      1.000122     -0.000122  
13     0.999756      1.000122      0.999939      0.000061  
Presione una tecla para continuar . . .
```